



CS 331 : Computer Networks
AY 2026-27

INTELLIGENT NETWORK CONFIGURATION ASSISTANT USING MCP AND RULE-BASED AUTOMATION

Safe automation for firewall and bandwidth configuration.

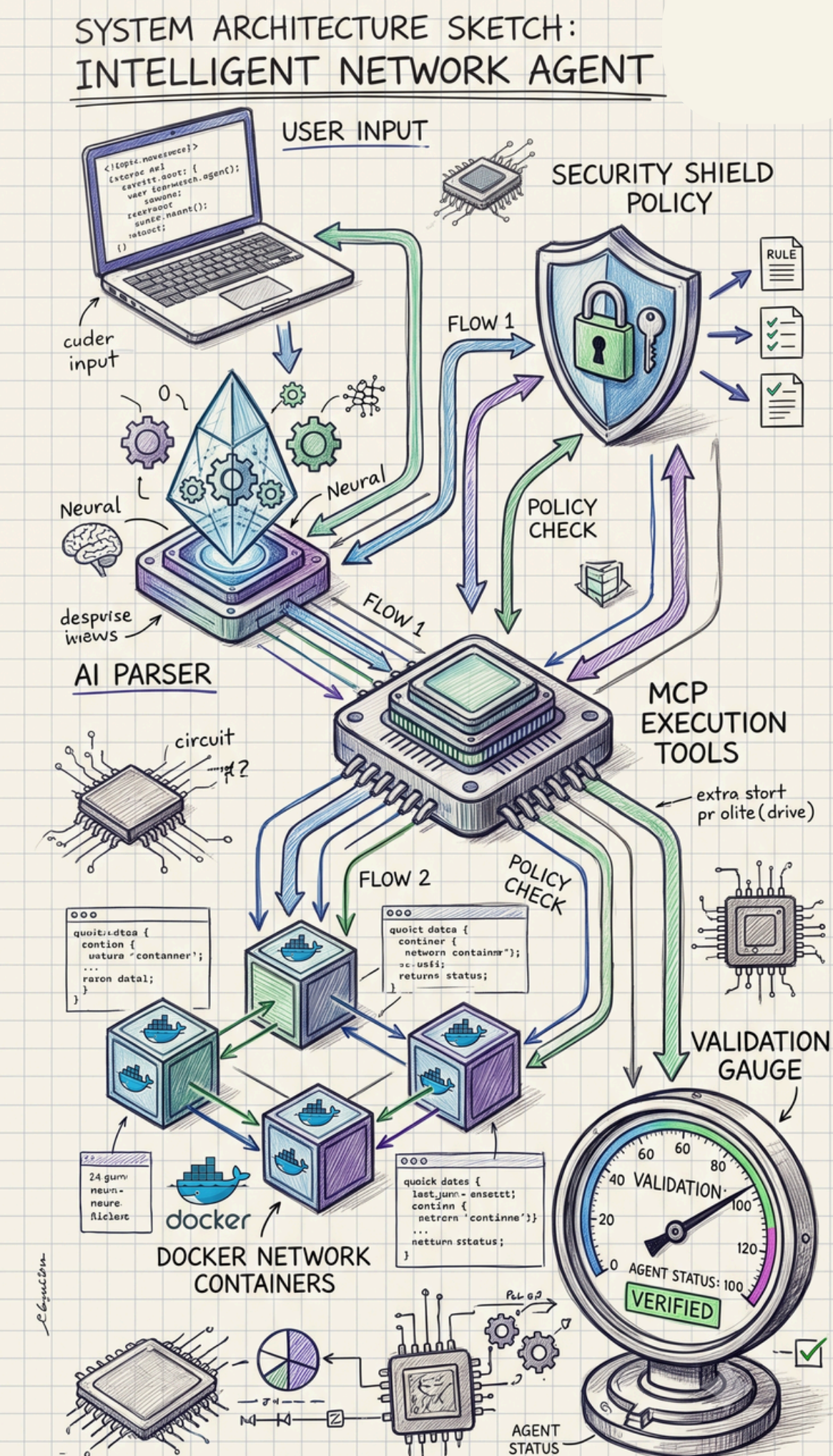
Team ID: T009 | Project ID: 5
Professor: Sameer G. Kulkarni
TA: Shubham Jaiswal

Ananya Patel (24110038)
Dhruv Meshram (24110109)
Khushi Garg (24110164)
Vedant Bondekar (24110384)

Indian Institute of Technology
Gandhinagar
Palaj, Gujarat - 382355

Equal contributions
Ordered based on roll number.

SOURCE:
[Source Link](#)



Safe network changes need a decision path before execution

Interpret the request, check safety rules, execute only an allowed action, then prove the result.

QUESTIONS WE ADDRESSED

- What is MCP, and why route actions through it instead of direct commands?
- How do we design a policy engine that can reliably ALLOW or DENY a request before any network change happens?
- How do we make sure the assistant can only take pre-approved, parameterized network actions (block, unblock, limit bandwidth) instead of arbitrary commands?
- Is our rule-based validation actually enforced at the network layer, or can it be bypassed by rephrasing a request?
- How do we prove a firewall or bandwidth change actually took effect on the network, instead of just trusting the command's exit status?

01

Interpret

What action, target, and parameters were requested?

02

Check

Can policy reliably ALLOW or DENY it?

03

Execute

Is it one pre-approved action, not an arbitrary command?

04

Verify

Did the live network state actually change?

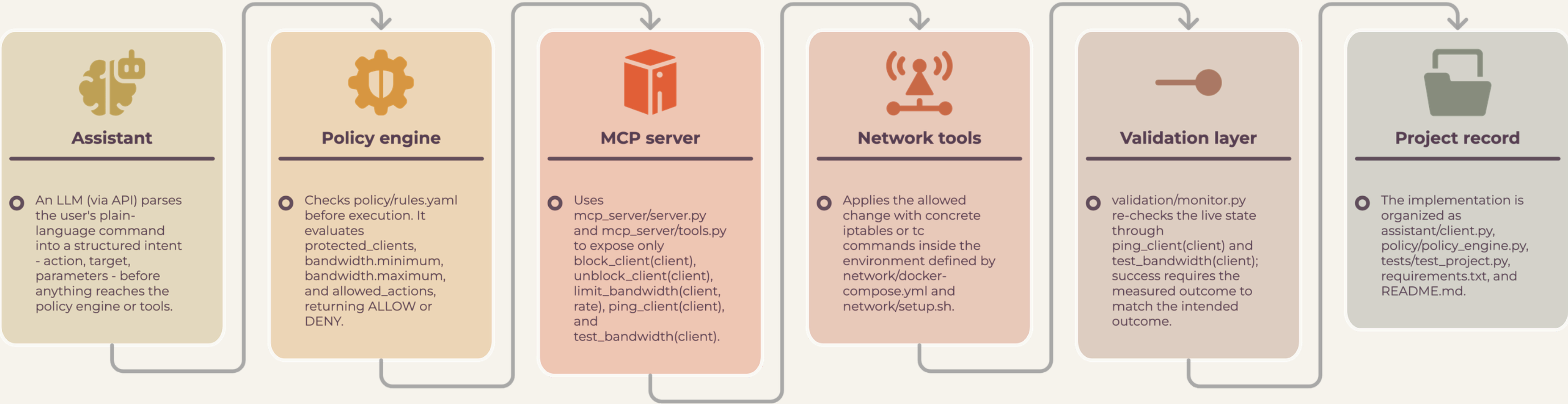
OUR WORKFLOW

1. Literature review of rule-based network automation and safe-control approaches.
2. Designed a YAML policy schema for protected clients, bandwidth limits, and allowed actions.
3. Built block_client, unblock_client, limit_bandwidth, ping_client, and test_bandwidth.
4. Set up an isolated Docker topology with Client1, Client2, and Server.
5. Used ping/iperf3 to benchmark actual network state against expected outcomes.

From plain-language request to measured network state

The architecture is deliberately layered because a command wrapper would simply automate the same unsafe assumptions.

Detailed Flow Diagram Process



Fail-safe principle: reject unsafe requests before execution, not after damage is possible. Validation closes the trust gap that simple automation leaves open.

Before vs after: limit a bandwidth hog

Step	Manual today	Through the assistant
Request	SSH in; recall the right tc syntax and interface.	"Limit client1 to 5 Mbps."
Safety	Human checks the target; no policy gate.	allowed_actions returns ALLOW for limit_bandwidth(client, rate).
Proof	Run ad hoc bandwidth tests and record results manually.	test_bandwidth(client) reports measured throughput.

We extended the basic version, then measured what really happened

The policy decision, real command, and observed network behavior must agree.

What we built on top

**Layer 2 filtering**

Used nftables at bridge level, so container-to-container traffic is caught even when it skips normal routing checks.

**Two-way limits**

Used an IFB virtual device to redirect incoming traffic, so downloads and uploads can use the same speed limit.

**Dynamic discovery**

Used the Docker SDK to find each container's current IP after restarts instead of using a fixed list.

**Audit trail**

Logged every allowed or denied decision with a timestamp.

Problems We Hit and How We Fixed Them

Problem	Why it happened	How we fixed it
Blocked device still replied to ping	The controller used a shortcut path that skipped filtering.	Pinged from another container on the same network.
Downloads stayed at full speed	Linux limits outgoing traffic by default, not incoming.	Redirected incoming traffic through a virtual device.
Limits remained after unblocking	Only the firewall rule was removed.	Added cleanup for all traffic-control settings.
Restarted container broke the system	The fixed IP list did not update.	Looked up each current container IP automatically.
Repeated tests hit “server busy”	The tool could get stuck after an unclean stop.	Restarted the testing tool when it was unresponsive.

Verified Results

Test	Result
Normal traffic before blocking	0% loss; full speed (~78 Gbps)
After blocking	100% loss; connection refused
After unblocking	0% loss; full speed restored (~77 Gbps)
5 Mbps upload limit	4.82 Mbps; within range
5 Mbps download limit	4.76 Mbps; within range
Automated tests	128 passed in about 11 seconds

Why this matters: ping and iperf3 independently measured the network after each command. We checked the real behavior, not just whether the command ran.

Project record: policy/rules.yaml defines protected clients, bandwidth limits, and allowed actions. Results are ALLOW or DENY with timestamps.

Non-functional testing turns “it ran” into evidence

Each check asks a plain question and records the observed result.

Parameter	What we tested	Result
Performance	How fast and accurate is bandwidth limiting?	Speeds stayed within 4–6% of the target across 2, 5, and 10 Mbps. This is inside the 20% tolerance.
Reliability	Does it detect both success and failure?	Yes. Blocking caused 100% loss; unblocking restored full speed every time.
Scalability	Does it depend on hardcoded device information?	No. Addresses are looked up automatically after devices restart or change.
Security	Can wording bypass the rules?	No. A separate safety check enforces the rules, including blocking commands hidden in device names.
Automation coverage	How thoroughly is the system tested?	128 automated tests cover discovery, safety, validation, and the full request flow. 100% passed.
Recovery / cleanup	Does the network end in a clean state?	Yes. Unblocking removes both the block and leftover speed limits.

Five challenges made the system safer and more reliable

Each problem exposed a gap between the command and the real network behavior.

Core learning: keep the AI's interpretation separate from the safety decision, execution, and proof.

1 · Blocked device still replied

Why: the test used a shortcut path that skipped filtering. **Solution:** pinged from another device on the same network, through the real path.

2 · Downloads were not limited

Why: Linux limits outgoing traffic by default. **Solution:** redirected incoming traffic through a virtual device so the same limit applied.

3 · Speed limits stayed after unblocking

Why: unblocking removed only the firewall rule. **Solution:** added cleanup that removes every speed-limit setting too.

4 · Container restarts broke the system

Why: the fixed IP list did not update. **Solution:** fetched the current address automatically every time.

5 · AI could misunderstand a request

Why: trusting the AI alone is unsafe when networks are affected. **Solution:** added an independent safety check that the AI cannot control, so unsafe actions remain blocked.